第十四届“恩智浦”杯全国大学生
智能汽车竞赛

技 术 报 告



学校：青岛科技大学

队伍名称：TSZ

参赛队员：马立栋 孙小童 丁佰勇

带队教师：刘海波 邢关生

关于技术报告和研究论文使用授权的说明

本人完全了解第 14 届“恩智浦”杯全国大学生智能汽车竞赛关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会和恩智浦半导体公司可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：

带队教师签名：

日　　期：2019/8/19

目 录

第一章 绪论.....1
1.1 智能车研究背景.....1
1.2 方案总体介绍.....1
1.3 本文结构.....1
第二章 机械结构设计及调整.....2
2.1 整车布局.....2
2.2 重心选择.....2
2.3 转向舵机安装.....3
第三章 硬件电路设计.....4
3.1 单片机系统设计.....4
3.2 电源模块设计.....4
3.3 电机驱动电路.....5
3.5 传感器的选择.....6
3.5.1 摄像头.....6
3.5.2 编码器.....6
第四章 软件控制设计.....7
4.1 赛道中心线提取及优化处理.....7
4.1.1 原始图像的特点.....7
4.1.2 赛道搜线算法.....8
4.2 PID 控制算法介绍.....10
4.2.1 位置式 PID.....11
4.2.2 增量式 PID.....11
4.2.3 PID 参数整定.....12
4.3 转向舵机的 PID 控制算法.....12
4.4 驱动电机的 PID 控制算法.....12
第五章 系统调试.....13
5.1 开发调试工具.....13
5.2 人机交互工具.....13
参考文献.....14
附录.....15

摘要

本文详细介绍了青岛科技大学“TSZ”在第十四届“恩智浦”杯全国大学生智能汽车竞赛光电四轮组中的系统方案。本次比赛采用大赛组委会统一指定的 C 型车模，以飞思卡尔半导体公司生产的 32 位单片机 MK66FX1M0VLQ18 为核心控制器，要求智能车识别道路两边黑线，以最快速度完成整个比赛，智能车采用 CMOS 摄像头对赛道信息检测，根据黑色边线提取赛道中线；通过编码器检测智能车的实时速度，使用 PID 控制算法调节电机的转速和舵机的打角，实现智能车在运动过程中速度和方向的闭环控制；为了提高模型车的速度和稳定性，使用上位机、按键 OLED 模块等调试工具，进行了大量硬件与软件测试。实验结果表明，该系统设计方案确实可行。

关键词: 智能车；MK66FX1M0VLQ18；图像处理；PID；

第一章 绪论

1.1 智能车研究背景

智能车是一种高新技术密集型的新型汽车，以汽车电子为背景涵盖控制、模式识别、传感技术、电子、电气、计算机、机械等多科学的科技创意性设计，一般主要由路径识别、速度采集、角度控制及车速控制等模块组成。可实现自动驾驶、自动变速及自动识别道路等功能，是现代电子产业发展中一项重要的组成部分。

在我国，教育部为了加强大学生实践、创新能力和团队合作精神的培养，委托教育部高等学校自动化专业教学指导分委员会主办了每年一度的全国大学生智能汽车竞赛。全国大学生智能汽车竞赛是在竞赛组委会提供的统一汽车模型平台上，使用飞思卡尔半导体公司的 8 位、16 位、32 位微控制器作为核心控制模块，通过设计道路识别传感器和电机驱动电路、编写相应软件及装配模型车，制作一个能够自主识别道路的模型汽车，按照规定路线（路线赛前未知）行进，以完成时间最短者为优胜。

1.2 方案总体介绍

比赛跑道为表面白色，两边有连续黑线作为引导线，黑线宽为 25mm。车模通过采集赛道图像进行路径检测比赛规则限定了跑道宽道区宽度 40cm，拐角最小半径 50cm，并且规定了各赛道标志的具体指标。参赛队员的目标是使智能车按照规则以最短时间完成单圈赛道。

根据竞赛规则相关规定，本智能车系统采用大赛组委会统一提供的 B 型车模，以恩智浦公司生产的 32 位微控制器 MK66FX1M0VLQ18 作为核心控制器，在 IAR 开发环境中进行软件开发。通过 CMOS 图像传感器拍摄赛道图像，将图像信号输入到 MK66FX1M0VLQ18 微控制器，进行进一步处理获得主要的赛道信息；通过编码器来检测车速，并通过单片机进行正交解码进行脉冲计算获得速。度和路程；转向舵机采用 PD 控制；驱动电机采用 PID 控制，通过 PWM 控制驱动电路调整电机的功率；而车速的目标值由默认值、运行安全方案和基于图像处理的优化策略进行综合控制。

1.3 本文结构

本文共分为六章。第一章主要是介绍了比赛的背景及智能车系统总体方案的介绍；第二章从智能车系统的机械结构出发，详细阐述了智能车系统各部分机械结构的安装和调整；第三章重点介绍了系统中所涉及的硬件设计方案和原理；第四章是介绍了智能系统的软件算法包括图像处理以及电机舵机的控制策略；第五章对调试过程中的一些手段进行了讲解；第六章则是对智能车一些物理参数进行了简单的汇报。

第二章 机械结构设计及调整

智能车的核心是控制策略和算法，但是，机械结构也是限制赛车速度的巨大瓶颈，如果一辆赛车的程序架构很好，但是机械部分做的不好的话，其速度也会被大大的限制。即当车速较高的时候，车模有明显的甩尾和侧滑现象，此时对车的机械结构要求很高。因此，我们在不违反规则的情况下对小车进行了多方面的改造以使小车具有良好的运行性能。

2.1 整车布局

本次竞赛选用北京博斯威龙科技有限公司生产的智能车竞赛专用模型车(C型模型车)，配套的电机为 380 电机，伺服器为 S3010。智能车的控制采用的是前轮转向，后轮驱动方案。智能车的外形如图 2.1 所示。

(1) 车模底盘降低，用来降低重心。
(2) 舵机采用竖直姿态，方便控制。
(3) 对前轮倾角进行调节，保证车直线行驶的稳定性。
(4) 电池后移，使重心尽量位于车体中后方，保证车在高速行进入弯时的稳定性
(5) 用轻便坚固的碳纤杆作为摄像头传感器的支撑材料。



图 2.1 智能车外形图

2.2 重心选择

在保证车模顺利通过坡道的前提下，尽可能的降低车子的重心。同时均匀车身重量，使重心在整车的中轴线上。由于靠前的重心会造成舵机负担，过后的重心又会导致侧滑。经过多次测试，我们确定了车重心的位置，并且将车模周围防护架拆除，电池采用自锁带固定于车身底盘如图 2.2 所示。

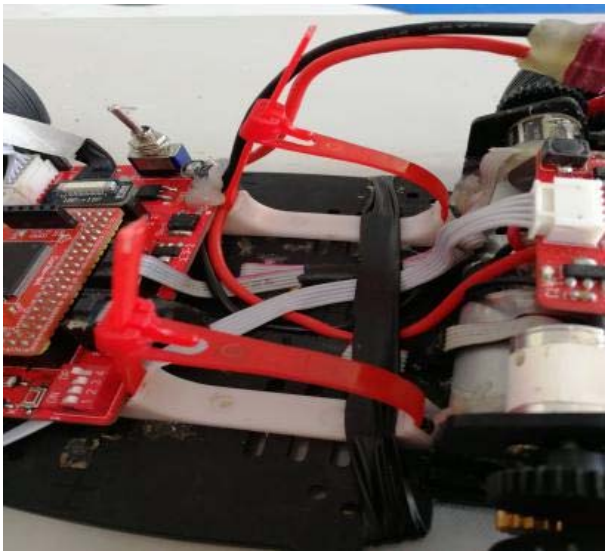


图 2.2 车整体固定与电池安装

2.3 转向舵机安装

舵机转向是整个控制系统中延迟较大的一个环节，为了减小此时间常数，通过改变舵机的安装位置可以提高舵机的响应速度。通过分析舵机控制转向轮转向的原理可以发现，在相同的舵机转向条件下，转向连杆在舵机一端的连接点离舵机轴心距离越远，转向轮转向变化越快。这相当于增大力臂长度，提高线速度。

针对上述特性，本设计采用舵机竖直放在前面，安装更长的舵机输出臂。采用杠杆原理，在舵机的输出舵盘上安装一个较长的输出臂，将转向传动杆连接在输出臂末端。这样就可以在舵机输出较小的转角下，取得较大的前轮转角，从而提高了整个车模转向控制的速度。如图 2.3 所示。同时舵机安装在车体前方正中央，使左右转角基本一致，避免了多级传动存在的空程和死区问题。

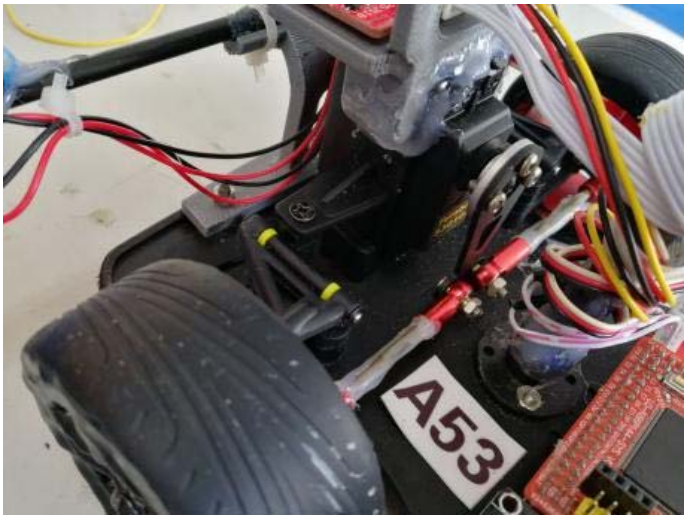


图 2.3 转向舵机安装

第三章 硬件电路设计

硬件是基础，只有一个良好、稳定的硬件环境才能保证车能平稳快速的行驶。我们在整个系统设计过程中严格按照规范进行。本着可靠、高效的原则，在满足各个要求的情况下，尽量使设计的电路简单。

3.1 单片机系统设计

单片机最小系统是智能车系统的核心控制部件。我们采用了 M4 内核 32 位的 K66 芯片。原理图如图 3.1 所示：

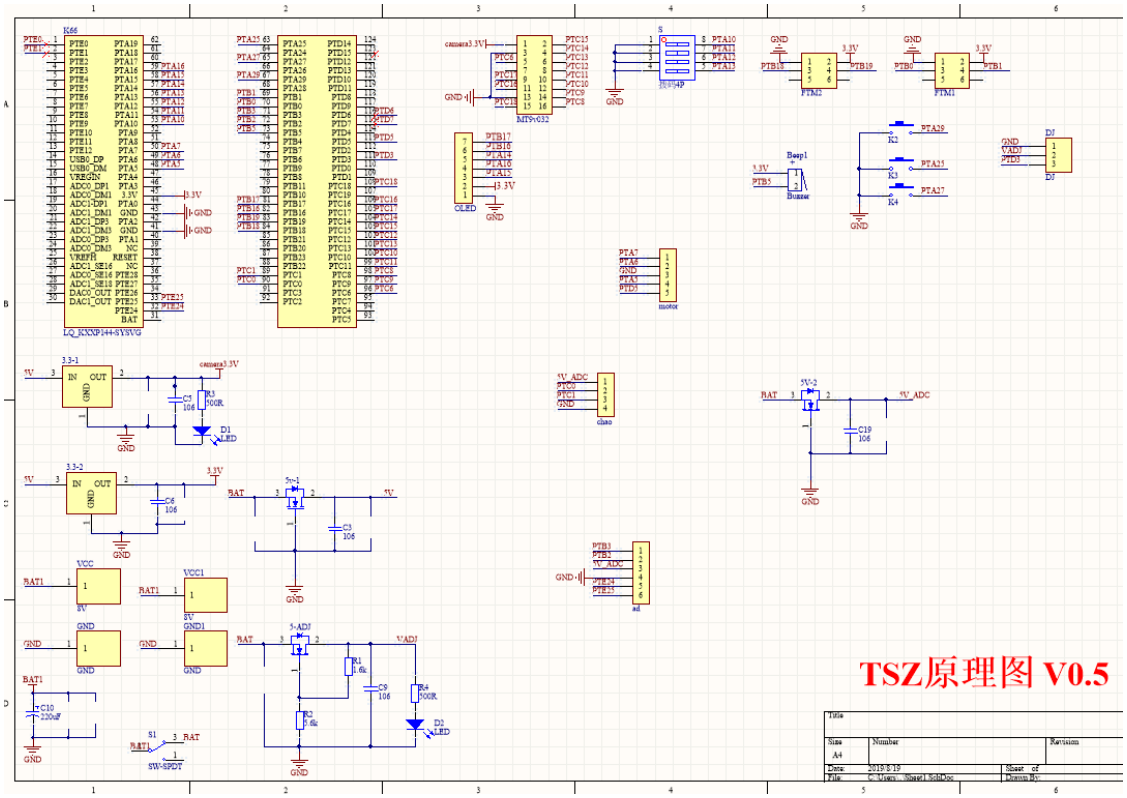


图 3.1 单片机系统原理图

3.2 电源模块设计

电源模块电源模块为系统其他各个模块提供所需要的电源。设计中，除了需要考虑电压范围和电流容量等基本参数之外，还要在电源转换效率、降低噪声、防止干扰和电路简单等方面进行优化。可靠的电源方案是整个硬件电路稳定可靠运行的基础。全部硬件电路的电源由配发的标准车模用 7.2V 2000mAh Ni-cd 镍镉充电提供。由于电路中的不同电路模块所需要的工作电压和电流容量各不相同，因此电源模块应该包含多个稳压电路，将充电电池电压转换成各个模块所需要的电压。为满足需要，本车模上存在 4 种供电电压：

(1)智能车使用镍镉充电电池，充满时电压在 7.8～8.2V。可直接用于电机以及舵机驱动供电。

(2)使用稳压芯片 AMS1084 输出电压 5V，用于红外、陀螺仪、蓝牙模块等供电。原理图如图 3.2 所示。

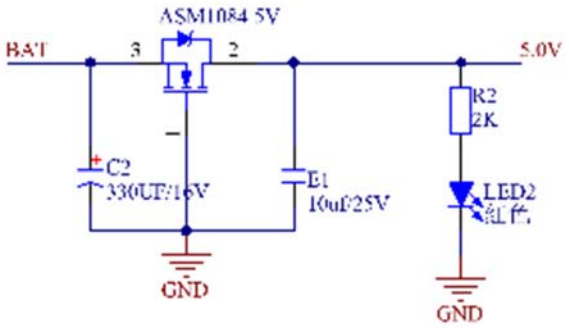


图 3.2 转压电路 7.2V-5V 原理图

(3)使用稳压芯片 AMS1117 输出电压 3.3V，用于 OLED、蜂鸣器、编码器、摄像头、单片机等供电。

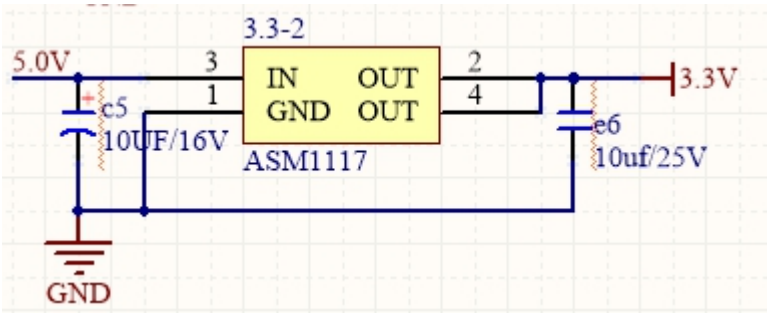


图 3.3 转压电路 5V-3.3V 原理图

(4) 使用稳压芯片 MC34063 输出电压 12V，用于半桥驱动器 IR2104 的供电。

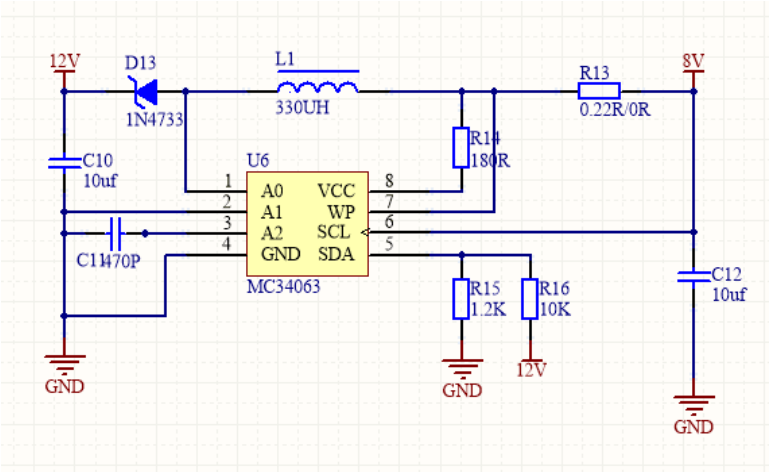


图 3.4 转压电路 7.2V-12V 原理图

3.3 电机驱动电路

在栅极驱动芯片选择方面，我们选择 IR2104 芯片，IR2104 芯片可以驱动高端和低端两个沟道 MOSFET，提供较大的驱动电流，并有硬件死区，防止同桥臂导通。使用两片 IR2104 可以构成一个 MOS 管全桥驱动电路，如图 3.5 所示。

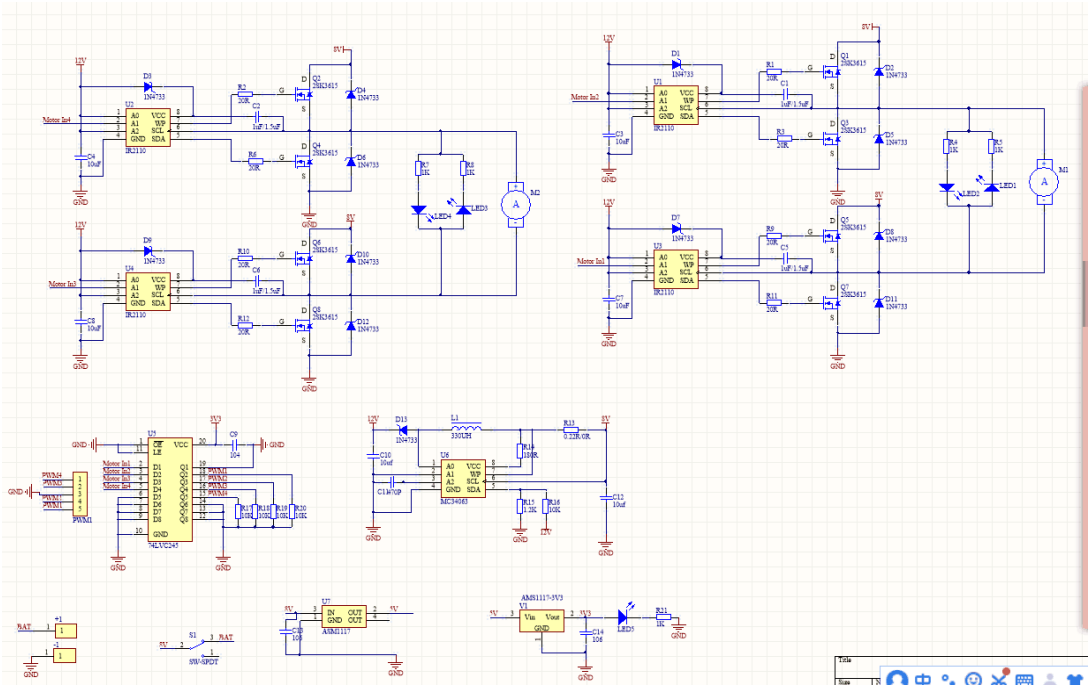


图 3.5 电机驱动原理图

3.5 传感器的选择

3.5.1 摄像头

比赛所用的摄像头可以分为两类：一类为 CCD 摄像头，另一类为 CMOS 摄像头。

CCD 摄像头图像对比度高、动态特性好，但供电电压比较高，需要 12V 的工作电压。在智能车的实际运行中，电机加减速时会产生很大的冲击电流，会对 12V 的升压模块造成冲击。同时 CCD 摄像头的耗电也比较严重，这会使拍摄的图像稳定性不高。CMOS 摄像头，体积小，图像稳定性较高，只需 3.3V 供电，耗电量低，但动态性能不如 CCD 摄像头好。智能车高速运行时，摄像头拍摄的图像可能会变得模糊。但为了保证系统的稳定性，最终决定选用 CMOS 摄像头。

3.5.2 编码器

光电编码器是一种通过光电转换将输出轴上的机械几何位移量转换成脉冲或数字量的传感器，这也是目前应用最多的测速传感器之一。其获取信息准确、精度高、应用简单。采用增量式 512 线光电编码器，其供电电压为 3.3V，输出为小幅值的正弦信号。为了将此信号放大整形，设计了信号调理电路，其基本原理是使用一个运放做成比较器电路，调节参考电压，使输出变为 0V-3.3V 的方波信号，送入单片机进行运算。

第四章 软件控制设计

高效的软件程序是智能车高速平稳自动寻线的基础。我们设计的系统采用 CMOS 摄像头进行赛道识别，图像采集及处理是整个软件的核心内容。在智能车的转向和速度控制方面，我们使用 PID 控制算法，使智能车能够稳定快速在赛道中循迹并完成比赛。

4.1 赛道中心线提取及优化处理

4.1.1 原始图像的特点

在单片机采集图像信号后需要对其进行处理以提取主要的赛道信息，同时，由于交叉道、起跑线的存在，光线不均匀、赛道外反光点、赛道远处图像不清楚的干扰，图像效果会大打折扣。因此，在软件上必须排除干扰因素，对赛道进行有效识别，并提供尽可能多的赛道信息供决策使用。

在图像信号处理中我们提取的赛道信息主要包括：赛道位于图像的位置、赛道边界点信息，赛道宽度信息，赛道中心线位置，赛道类型识别。由于摄像头自身的特性和远小近大的视觉原理，图像会产生梯形式变形，这使得摄像头看到的赛道信息和实际真实的赛道信息有所区别。因此我们利用赛道进行测量和标定，得到一系列的参数，将摄像头采到的原始图像还原出真实赛道信息。原始图像是一个将模拟图像经模拟电路转换得到的二维数据矩阵，矩阵的每一个元素对应一个像素点，图像的第一行对应最远处，距摄像头固定杆大约 160cm，图像的最底部一行对应最近处，距固定杆 5cm。远处的图像小，近处的图像大，黑线为梯形状。

摄像头返回的这个矩阵中，每一个像素点都有一个从 0 至 255 的灰度值，值越大表示该像素点越亮，相反，越暗的像素点值就越小。同色间的灰度差很小，而不同色间的灰度差很大，我们使用固定阈值将像素矩阵二值化，得到黑白图像，将赛道与背景区别开来。我们对摄像头高度与俯角进行了标定，得到一系列参数，对原始的透视图像进行逆透视变换，得到了用于单片机处理的基本图像。

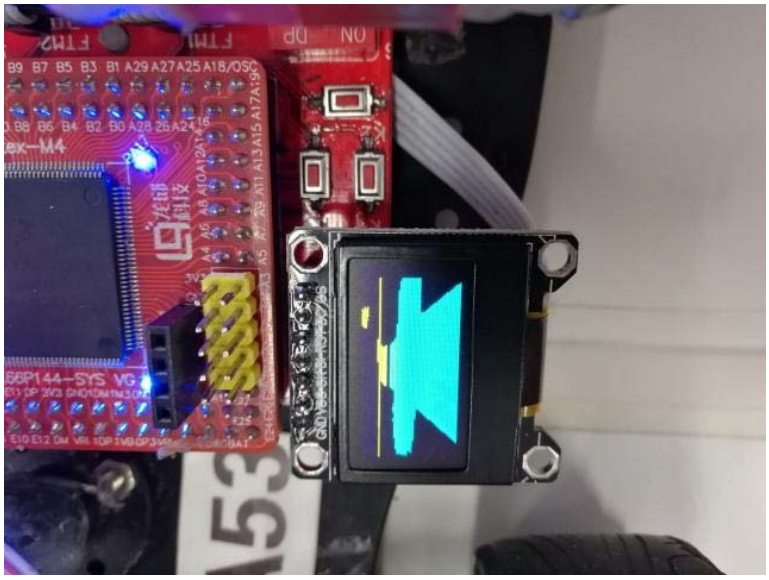


图 4.1 十字路原始图像



图 4.2 起跑线原始图像

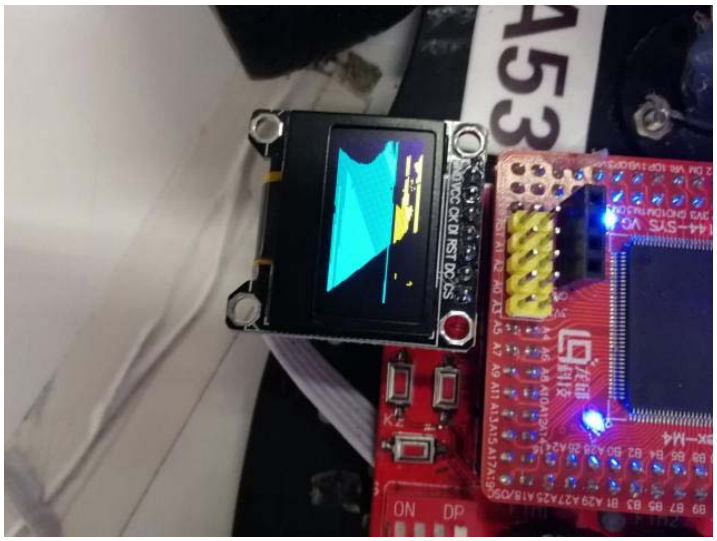


图 4.3 环岛原始图像

4.1.2 赛道搜线算法

单片机在对原始图像进行二值化之后，用搜线算法来对其进行处理，从而确定出赛道两边的黑线在哪儿，并计算出相对应的赛道中线，从而指导舵机转向和电机的加减速。搜线算法的基本思想如下：

(1)首先搜索左边线，左边线搜完之后再搜索右边线 ；

(2)由图像中心向左一定范围后开始逐点往左边搜索，具体的搜索方式是相邻点灰度值作差，当差值大于某一个阈值时，表明灰度发生了跳变，即是搜索到了黑线，将该行存入对应的存储矩阵，跳入下一行，重复步骤（2）；

(3)如果连续几行都搜索到黑线后，将通过斜率预测的方法确定下一行黑线所在位置的范围，从而减少搜索的计算量。具体的方法即计算出前几行黑线所在点连成直线的斜率，从而延长直线与下一行相交，得到预测到的黑线位置。该过程将一直重复，直到出现某行搜索不到黑线或者搜索到图像尽头。

(4)当某行搜索不到黑线时，将启动虚线搜索的算法，同样是通过斜率预测，但是是通过隔 3 行扫描一个黑线。当连续几行搜索黑线后，再次进入连续斜率预测

的搜索算法

(5)当左边线搜索图像底部之后，将跳入右边线的搜索。搜线完毕后，再进行滤波处理，就得到了处理好的能够进行赛道分析的完整的矩阵。

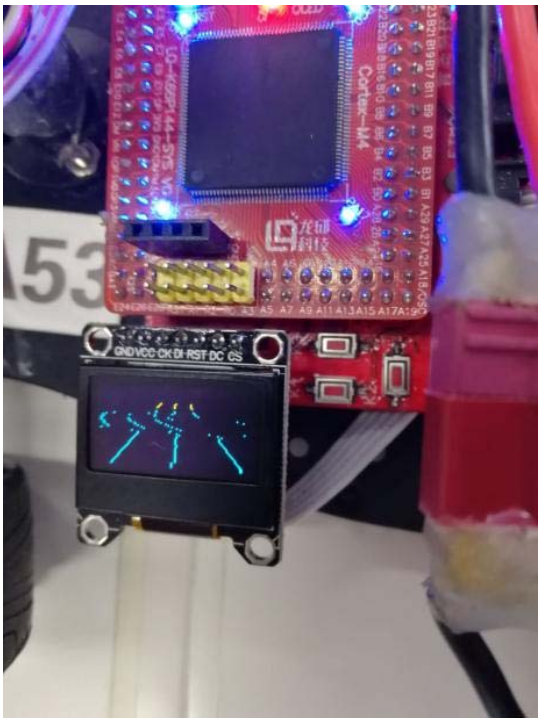


图 4.4 十字边缘搜线图

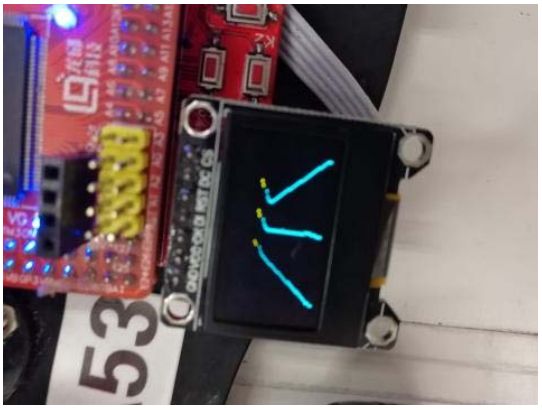


图 4.5 起跑线边缘搜线图

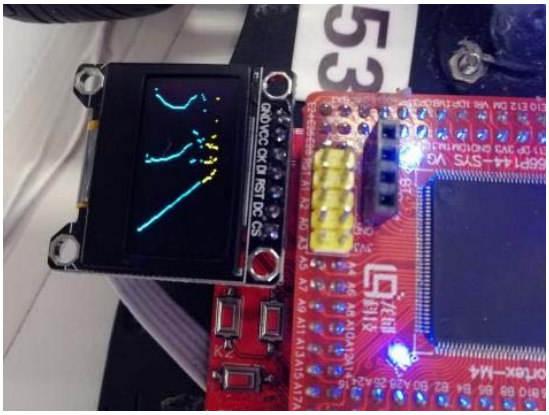


图 4.6 环岛边缘搜线图

4.2 PID 控制算法介绍

在工程实际中，应用最为广泛的调节器控制规律为比例、积分、微分控制，简称 PID 控制，又称 PID 调节。PID 控制器问世至今已有近 70 年历史，它以其结构简单、稳定性好、工作可靠、调整方便而成为工业控制的主要技术之一。当被控对象的结构和参数不能完全掌握，或得不到精确的数学模型时，控制理论的其它技术难以采用时，系统控制器的结构和参数必须依靠经验和现场调试来确定，这时应用 PID 控制技术最为方便。即当我们不完全了解一个系统和被控对象，或不能通过有效的测量手段来获得系统参数时，最适合用 PID 控制技术。PID 控制，实际中也有 PI 和 PD 控制。

PID 控制器是一种线性控制器，它根据给定值与实际输出值构成控制偏差。将偏差的比例(P)、积分(I)和微分(D)通过线性组合构成控制量，对被控对象进行控制，故称 PID 控制器，原理框图如图 4.7 所示。

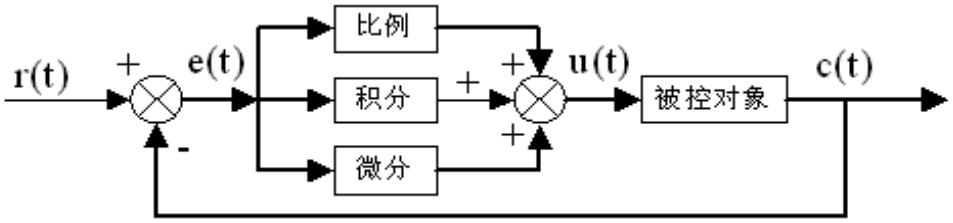


图 4.7 PID 控制器原理框图

在计算机控制系统中，使用的是数字 PID 控制器，控制规律为：

$$e(k)=r(k)-c(k) \tag{公式 4.1}$$

$$u(k)=K_P\{e(k)+\frac{T}{T_I}\sum_{j=0}^{k}e(j)+\frac{T_D}{T}[e(k)-e(k-1)]\} \tag{公式 4.2}$$

式中：

k ——采样序号，k = 0，1，2…；　　$r(k)$ ——第 k 次给定值；

$c(k)$ ——第 k 次实际输出值；　　$u(k)$ —— 第 k 次输出控制量；

$e(k)$ —— 第 k 次偏差；　　$e(k-1)$ —— 第 k-1 次偏差；

K_P ——比例系数；　　T_I ——积分时间常数；

T_D ——微分时间常数；　　T ——采样周期。

简单说来，PID 控制器各校正环节的作用如下：

比例环节：及时成比例地反映控制系统的偏差信号，偏差一旦产生，控制器立即产生控制作用，以减少偏差。

积分环节：主要用于消除静差，提高系统的无差度。积分作用的强弱取决于积分时间常数，越大，积分作用越弱，反之则越强。

微分环节：能反映偏差信号的变化趋势(变化速率)，并能在该偏差信号变得太大

之前，在系统中引入一个有效的早期修正信号，从而加快系统的动作速度，减小调节时间。

数字 PID 控制算法通常分为位置式 PID 控制算法和增量式 PID 控制算法。

4.2.1 位置式 PID

位置式 PID 中，由于计算机输出的$u(k)$直接去控制执行机构(如阀门)，$u(k)$的值和执行机构的位置(如阀门开度)是一一对应的，所以通常称公式(5.2)为位置式 PID 控制算法。

位置式 PID 控制算法的缺点是：由于全量输出，所以每次输出均与过去的状态有关，计算时要对过去$e(k)$进行累加，计算机工作量大；而且因为计算机输出的$u(k)$对应的是执行机构的实际位置，如计算机出现故障，$u(k)$的大幅度变化，会引起执行机构位置的大幅度变化，这种情况往往是生产实践中不允许的，在某些场合，还可能造成严重的生产事故。因而产生了增量式 PID 控制的控制算法，所谓增量式 PID 是指数字控制器的输出只是控制量的增量$\Delta u(k)$。

4.2.2 增量式 PID

当执行机构需要的是控制量的增量(例如：驱动步进电机)时，可由式(4.2)推导出提供增量的 PID 控制算式。由式(4.2)可以推出式(4.3)，式(4.2)减去式(4.3)可得式(4.4)。

$$u(k-1)=K_P\{e(k-1)+\frac{T}{T_I}\sum_{j=0}^{k-1}e(j)+\frac{T_D}{T}[e(k-1)-e(k-2)]\} \quad \text{(公式 4.3)}$$

$$\Delta u(k)=K_P\{[e(k)-e(k-1)]+\frac{T}{T_I}e(k)+\frac{T_D}{T}[e(k)-2e(k-1)+e(k-2)]\}$$

$$=K_P\Delta e(k)+K_I e(k)+K_D[\Delta e(k)-\Delta e(k-1)]\} \quad \text{(公式 4.4)}$$

式中$\Delta e(k)=e(k)-e(k-1)$；$K_I=K_P\frac{T}{T_I}$；$K_D=K_P\frac{T_D}{T}$。

公式(4.4)称为增量式 PID 控制算法，可以看出由于一般计算机控制系统采用恒定的采样周期T，一旦确定了K_P、T_I、T_D，只要使用前后三次测量值的偏差，即可由式(4.4)求出控制增量。

增量式 PID 具有以下优点：

(1) 由于计算机输出增量，所以误动作时影响小，必要时可用逻辑判断的方法关掉。

(2) 手动/自动切换时冲击小，便于实现无扰动切换。此外，当计算机发生故

障时，由于输出通道或执行装置具有信号的锁存作用，故能保持原值。

(3) 算式中不需要累加。控制增量$\Delta u(k)$的确定仅与最近 k 次的采样值有关，所以较容易通过加权处理而获得比较好的控制效果。
但增量式 PID 也有其不足之处：积分截断效应大，有静态误差；溢出的影响大。使用时，常选择带死区、积分分离等改进 PID 控制算法。

4.2.3 PID 参数整定

运用 PID 控制的关键是调整K_P、K_I、K_D三个参数，即参数整定。PID 参数的整定方法有两大类：一是理论计算整定法。它主要是依据系统的数学模型，经过理论计算确定控制器参数；二是工程整定方法，它主要依赖工程经验，直接在控制系统的试验中进行，且方法简单、易于掌握，在工程实际中被广泛采用。由于智能车系统本身是一个机电高度结合且相互依赖的系统，并且还需要考虑赛道具体环境带来的影响，要建立精确的智能车运动控制数学模型有一定难度，而且我们对小车的机械结构修改也会带来参数的频繁变化，故理论计算整定法的操作性不强，最终我们采用了工程整定方法。

4.3 转向舵机的 PID 控制算法

对于舵机的闭环控制，我们采用了位置式 PID 控制算法。在参考了往届优秀技术报告中使用的舵机控制方法之后，我们在他们的基础上做出了自己改进以及完善。结合我们的路况提取算法，通过传统的 PID 算法对舵机进行控制，去得了令人满意的效果。

4.4 驱动电机的 PID 控制算法

速度控制对于小车的快速行进以及必要时候的减速起到了至关重要的作用，我们在尝试使用传统 PID 方法之后，发现并不能达到我们理想的速度控制需求，于是对其进行了自己的改进，加入了额外项进行修正，取得了不错的效果。

第五章 系统调试

5.1 开发调试工具

软件开发工具选用的是 Embedded Workbench for ARM。是 IAR Systems 公司为 ARM 微处理器开发的一个集成开发环境(下面简称 IAR)。比较其他的 ARM 开发环境，IAR 具有入门容易、使用方便和代码紧凑等特点。它为用户提供一个易学和具有最大量代码继承能力的开发环境，以及对大多数和特殊目标的支持。嵌入式 IAR Embedded Workbench 有效提高用户的工作效率，通过 IAR 工具，可以大大节省软件调试时间。

5.2 人机交互工具

在调试过程中需要不断地修改变量的值来达到整定参数的作用，对此我们选用了液晶屏配合按键和拨码开关的调试方法。此外，比赛的时候，修改参数我们同样用这个模块进行修改。

参考文献

[1] 卓晴，黄开胜，邵贝贝．学做智能车 [M]．北京：北京航空航天大学出版社．2007.
[2] 王淑娟，蔡惟铮，齐明．模拟电子技术基础 [M]．北京：高等教育出版社．2009
[3] 张军．AVR 单片机应用系统开发典型实例 [M]．北京：中国电力出版社，2005.
[4] 张文春．汽车理论 [M]．北京．机械工业出版社．2005.
[5] 殷剑宏，吴开亚．图论及其算法 [M] ．中国科学技术大学出版社，2003.
[6] 夏克俭．数据结构及算法 [M] ．北京：国防工业出版社， 2001.
[7] 邵贝贝．单片机嵌入式应用的在线开发方法 [M]．北京．清华大学出版社．2004.
[8] 蔡述庭．“飞思卡尔”杯智能汽车竞赛设计与实践 [M]．北京：北京航空航天大学出版社.2012.

附录

基础寻线算法

```
void get_route(void)
{
  //基础寻线
  get_route_one(Row-1,Col/2);
  get_route_all();
  //赛道信息提取
  GetBlackEndParam();
  //截止行判断
  get_route_endline();
```

```
if(ringflag==0&&cutrouteflag==0&&obstructflag==0&&rampflag==0&&asternflag==0
&&scratchflag==0)
  {
    //坡道  时间微调
    scratch_line();
  }
```

```
if(ringflag==0&&cutrouteflag==0&&obstructflag==0&&rampflag==0&&asternflag==0
&&scratchflag==0)
  {
    //坡道  时间微调
    ramp_find();
  }

if(ringflag==0&&obstructflag==0&&rampflag==0&&asternflag==0&&scratchflag==0)
  {
    //横断  阈值微调
    obstruct_find_plain();
    if(obstructflag==1)
      cutrouteflag=0;
  }

if(ringflag==0&&cutrouteflag==0&&obstructflag==0&&rampflag==0&&asternflag==0
&&scratchflag==0)
  {
    //断路
    cutroute_find();
```

```
    }

if(ringflag==0&&cutrouteflag==0&&obstructflag==0&&rampflag==0&&asternflag==0
&&scratchflag==0)
    {
        //圆环
        ring_find();
    }

if(ringflag==0&&cutrouteflag==0&&obstructflag==0&&rampflag==0&&scratchflag==
0&&asternflag==0&&scratchflag==0)
    {
        //十字
        cross_find();
    }

    //全元素开响
    gpio_set(B5,1);

if(obstructflag!=0||cutrouteflag!=0||ringflag!=0||scratchflag==1||xie_cross_flag==1
||ru_cross_flag==1||chu_cross_flag==1||rampflag!=0)
        gpio_set(B5,0);

    //元素处理
    ramp_deal();
    cross_deal();
    ring_start();
    ring_ru();
    ring_chu();
    obstruct_deal();
    cut_route_process();
    //拟合
    get_route_midline();
}
//第一行图像寻线
//起跑线与其他图像寻线方式区别
void get_route_one(uint8 hang,uint8 start)
{
    //清零
    for(uint8 hang=0;hang<Row;hang++)
    {
        leftline[hang]=0;
        rightline[hang]=Col-1;
        leftlineflag[hang]=0;
```

```
    rightlineflag[hang]=0;
    midline[hang]=Col/2;
}
endline=0;
//检测本行起跑线标志位
int baihei=0;
for(uint8 lie=0;lie<Col-1;lie++)
{
    if(Pixle[hang][lie]==white&&Pixle[hang][lie+1]==black)
        baihei++;
}
//起跑线寻线
if(baihei>3)
{
    if(Pixle[hang][5]==white)
    {
        //向左找
        for(uint8 lie=5;lie>=1;lie--)
        {
          if(Pixle[hang][lie]==white&&Pixle[hang][lie-1]==black)
          {
                leftline[hang]=lie;
                leftlineflag[hang]=1;
                break;
          }
        }
    }
    else
    {
        //向右找
        for(uint8 lie=5;lie<=Col-2;lie++)
        {
          if(Pixle[hang][lie]==black&&Pixle[hang][lie+1]==white)
          {
                leftline[hang]=lie;
                leftlineflag[hang]=1;
                break;
          }
        }
    }
    //如果没找到，直接给定值
    if(leftlineflag[hang]==0)
    {
        leftline[hang]=1;
```

```
        leftlineflag[hang]=1;
    }
    //右边线
    if(Pixle[hang][Col-5]==white)
    {
        //向右找
        for(uint8 lie=Col-5;lie<=Col-2;lie++)
        {
          if(Pixle[hang][lie]==white&&Pixle[hang][lie+1]==black)
          {
                rightline[hang]=lie;
                rightlineflag[hang]=2;
                break;
          }
        }
    }
    else
    {
        //向左找
        for(uint8 lie=Col-5;lie>=1;lie--)
        {
          if(Pixle[hang][lie]==black&&Pixle[hang][lie-1]==white)
          {
                rightline[hang]=lie;
                rightlineflag[hang]=1;
                break;
          }
        }
    }
    //如果没找到，直接给定值
    if(rightlineflag[hang]==0)
    {
        rightline[hang]=Col-2;
        rightlineflag[hang]=2;
    }
}
else
{
    //左边线
    if(Pixle[hang][start]==white)
    {
        //向左找
        for(uint8 lie=start;lie>=1;lie--)
        {
```

```
      if(Pixle[hang][lie]==white&&Pixle[hang][lie-1]==black)
      {
            leftline[hang]=lie;
            leftlineflag[hang]=2;
            break;
      }
    }
}
else
{
    //向右找
    for(uint8 lie=start;lie<=Col-2;lie++)
    {
      if(Pixle[hang][lie]==black&&Pixle[hang][lie+1]==white)
      {
            leftline[hang]=lie;
            leftlineflag[hang]=1;
            break;
      }
    }
}
//如果没找到，直接给定值
if(leftlineflag[hang]==0)
{
    leftline[hang]=1;
    leftlineflag[hang]=2;
}
//右边线
if(Pixle[hang][start]==white)
{
    //向右找
    for(uint8 lie=start;lie<=Col-2;lie++)
    {
      if(Pixle[hang][lie]==white&&Pixle[hang][lie+1]==black)
      {
            rightline[hang]=lie;
            rightlineflag[hang]=2;
            break;
      }
    }
}
else
{
    //向左找
```

```
        for(uint8 lie=start;lie>=1;lie--)
        {
          if(Pixle[hang][lie]==black&&Pixle[hang][lie-1]==white)
          {
                rightline[hang]=lie;
                rightlineflag[hang]=1;
                break;
          }
        }
      }
      //如果没找到，直接给定值
      if(rightlineflag[hang]==0)
      {
        rightline[hang]=Col-2;
        rightlineflag[hang]=2;
      }
      //如果左右边线都不绝对可信
      if(rightlineflag[hang]==1&&leftlineflag[hang]==1)
      {
        leftline[hang]=1;
        rightline[hang]=Col-2;
      }
      leftlineflag[hang]=1;
      rightlineflag[hang]=1;
    }
}
//跟踪寻线
void get_route_all(void)
{
   //初始化
   int Lstart=0,L_max=0,L_min=0;
   int Rstart=0,R_max=0,R_min=0;
   int range=10;
   for(uint8 hang=Row-2;hang>0;hang--)
   {
     //左边线
     //确定搜线范围
      //根据上一行确定本行寻线点
      if(leftlineflag[hang+1]==0)
      {
        L_max=Col-1;
        L_min=0;
        Lstart=Col*1/2-10;
      }
```

```
else
{
   L_max=leftline[hang+1]+range;
   L_min=leftline[hang+1]-range;
   Lstart=leftline[hang+1];
   if(L_max>Col-1)
      L_max=Col-1;
   if(L_min<0)
      L_min=0;
}
//左边线
if(Pixle[hang][Lstart]==black)
{
   //向右找
   for(uint8 lie=Lstart;lie<L_max;lie++)
   {
      if(Pixle[hang][lie]==black&&Pixle[hang][lie+1]==white)
      {
          leftline[hang]=lie;
          leftlineflag[hang]=1;
          break;
      }
   }
   //如果还没找到说明丢线
   if(leftlineflag[hang]==0)
   {
      leftline[hang]=Col-2;
      leftlineflag[hang]=0;
   }
}
else
{
   //向左找
   for(uint8 lie=Lstart;lie>L_min;lie--)
   {
      if(Pixle[hang][lie]==white&&Pixle[hang][lie-1]==black)
      {
            leftline[hang]=lie;
            leftlineflag[hang]=1;
          break;
      }
   }
   //如果还没找到说明丢线
   if(leftlineflag[hang]==0)
```

```
    {
        leftline[hang]=1;
        leftlineflag[hang]=0;
    }
}
//右边线
//确定寻线范围
if(rightlineflag[hang+1]==0)
{
    Rstart=Col*1/2+10;
    R_max=Col-1;
    R_min=0;
}
else
{
    Rstart=rightline[hang+1];
    R_max=rightline[hang+1]+range;
    R_min=rightline[hang+1]-range;
    if(R_max>Col-1)
        R_max=Col-1;
    if(R_min<0)
        R_min=0;
}
//右边线
if(Pixle[hang][Rstart]==black)
{
    //向左找
    for(uint8 lie=Rstart;lie>R_min;lie--)
    {
        if(Pixle[hang][lie]==black&&Pixle[hang][lie-1]==white)
        {
            rightline[hang]=lie;
            rightlineflag[hang]=1;
            break;
        }
    }
    //如果还没找到
    if(rightlineflag[hang]==0)
    {
        rightline[hang]=1;
        rightlineflag[hang]=0;
    }
}
else
```

```
        {
            //向右找
            for(uint8 lie=Rstart;lie<R_max;lie++)
            {
                if(Pixle[hang][lie]==white&&Pixle[hang][lie+1]==black)
                {
                    rightline[hang]=lie;
                    rightlineflag[hang]=1;
                    break;
                }
            }
            //如果还没找到
            if(rightlineflag[hang]==0)
            {
                rightline[hang]=Col-2;
                rightlineflag[hang]=0;
            }
        }
    }
    //车最远一行按上一行补
    leftline[0]=leftline[1];
    leftlineflag[0]=leftlineflag[1];
    rightline[0]=rightline[1];
    rightlineflag[0]=rightlineflag[1];
}
```